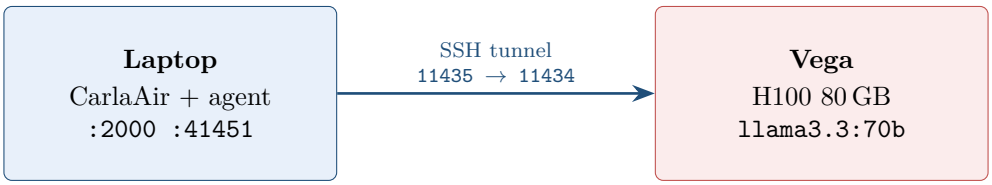
Connecting an HPC Language Model to a Local CarlaAir Simulator

A verified manual for ERAU Vega

Agentic UAV coordination study — open-loop baseline



Aditya Chikhaliya
Embry–Riddle Aeronautical University

Setup performed 6 October 2026 · revised 20 October 2026

Every command marked **[verified]** was run on Vega and its output observed.
Commands marked **[not exercised]** are documented but not yet exercised.

Contents

1 What this manual is **2**
1.1 Who it is for . . . 2
1.2 The problem being solved . . . 2

2 The architecture, and why it is sound **2**
2.1 What runs where . . . 2
2.2 Why splitting the system does not corrupt the measurements . . . 2
2.3 Why a model also runs on the laptop . . . 3

3 Vocabulary of high-performance computing **4**
3.1 The machine . . . 4
3.2 Getting work to run . . . 4
3.3 Networking . . . 5
3.4 Language models . . . 5

4 Step 1 — Getting in **6**
4.1 The network rules . . . 6
4.2 The VPN, in order . . . 6
4.2.1 Prove the route works before blaming SSH [verified] . . . 6
4.3 Logging in [verified] . . . 7

5 Step 2 — Storage, and where the model goes **7**

6 Step 3 — Installing Ollama without administrator rights **7**
6.1 Why it has to be installed by hand . . . 7
6.2 The documented download URL is broken [verified] . . . 8
6.3 Installing [verified] . . . 8
6.4 The environment file [verified] . . . 8

7 Step 4 — Staging the models **9**

8 Updating Ollama **9**
8.1 What was done, 19 October 2026 [verified] . . . 10
8.2 Confirming in the next GPU job [verified] . . . 12
8.3 Going back . . . 12

9 Step 5 — Getting a GPU **12**
9.1 What to ask for, and why 48 cores . . . 12
9.2 The queue situation, as actually found [verified] . . . 12
9.3 Reading the request . . . 13
9.4 Waiting, and finding out why [verified] . . . 13
9.5 Nodes are shared, so a modest request gets in sooner [verified] . . . 14
9.6 Before you ask: which node has an idle card? [verified] . . . 14
9.7 Asking for a specific node, and pointing the tunnel at it . . . 15

10 Step 6 — Which GPU is mine? **15**

11 Step 7 — Serving the model, and the memory trap **16**
11.1 Starting the server [verified] . . . 16
11.2 Finding the server again later . . . 17
11.3 Why the context length nearly ruined the experiment [verified] . . . 18

12 Step 8 — The SSH tunnel **18**
12.1 What a tunnel actually is . . . 18
12.2 Opening it [verified] . . . 19
12.3 Why 11435 and never 11434 — a data-integrity decision . . . 19

13 Step 9 — Wiring the agent, and refusing to fly blind **20**
13.1 The endpoint is explicit and recorded . . . 20
13.2 A pre-flight check that refuses to proceed . . . 20
13.3 Transport failures are not model failures . . . 20
13.4 Inference options are declared, not defaulted . . . 20

14 Step 10 — Proving it works, in order **20**
14.1 What the working system looked like [verified] . . . 21

15 Step 11 — Running the experiment, from the laptop **21**
15.1 Three things the laptop needs first . . . 21
15.1.1 The conda environment . . . 21
15.1.2 The simulator . . . 22
15.1.3 The tunnel . . . 22
15.2 The command . . . 22
15.2.1 Flags not used here, and when they matter . . . 23
15.3 What it then does, in order . . . 23
15.4 Expected timings . . . 23

16 Step 12 — The interactive session **24**
16.1 The idea . . . 24
16.2 What a session looks like . . . 24
16.3 The details that matter . . . 26
16.4 Two findings it produced on its first use . . . 26

17 Troubleshooting **28**

18 Shutting down properly **29**

19 Open questions **29**

A Command reference **29**
A.1 Laptop, before anything else . . . 29
A.2 On the login node . . . 30
A.3 Seeing the cluster . . . 30
A.4 Which cards are idle, on both GPU nodes . . . 30
A.5 Getting a GPU . . . 30
A.6 Updating Ollama (login node, nothing serving) . . . 30
A.7 On the GPU node . . . 31
A.8 From the laptop . . . 31
A.9 Shutting down . . . 31

1 What this manual is

This manual records how a large language model running on a university supercomputer was connected to a flight simulator running on a laptop, so that the simulator could ask the model to plan missions for uncrewed aerial vehicles.

It is written from a working installation rather than from documentation. Where the official documentation is wrong, this says so and gives what actually worked. Where something was never tested, it says that too, because a manual that cannot distinguish the two is not trustworthy.

The one-sentence version

Run `ollama serve` inside a GPU job on Vega bound to `0.0.0.0:11434`, open an SSH tunnel from the laptop that forwards local port **11435** to that node's 11434 through the login node, and point the agent at `http://127.0.0.1:11435`.

1.1 Who it is for

Someone who has an ERAU account, a laptop that runs the simulator, and no prior experience of high-performance computing. Every term is defined at first use and collected in the glossary in Section 3.

1.2 The problem being solved

The research question is how a language model's capability affects the quality of flight plans it produces. Answering it needs models of different sizes. A laptop with 4 GB of video memory can run a 3-billion-parameter model; a 70-billion-parameter model needs roughly 42 GB and will not fit. Vega has H100 GPUs with 80 GB each.

So the model must move to the cluster. The simulator cannot: CarlaAir is a Windows build that needs a graphics display, and the cluster is a headless Linux system. The two halves therefore have to run on different machines and talk to each other, which is what the rest of this manual is about.

2 The architecture, and why it is sound

2.1 What runs where

2.2 Why splitting the system does not corrupt the measurements

This is the part that makes the architecture defensible rather than merely convenient.

The baseline agent is **open loop**. It asks the model for a complete plan *before the aircraft leaves the ground*, validates that plan against a schema, and then executes it without ever consulting the model again. So:

- exactly one network request happens per drone per mission;
- it happens while the aircraft is stationary on the ground;
- network latency therefore cannot affect flight timing, control stability, or the separation between drones.

What is *not* comparable across machines

Plan validity, exact-sequence correctness, consistency and raw model output are properties of the model and compare directly between laptop and cluster. **Planning latency does not** — a laptop CPU and a shared cluster GPU plus a VPN hop are not the same measurement. Flight time, collisions and landing are unaffected, being entirely local.

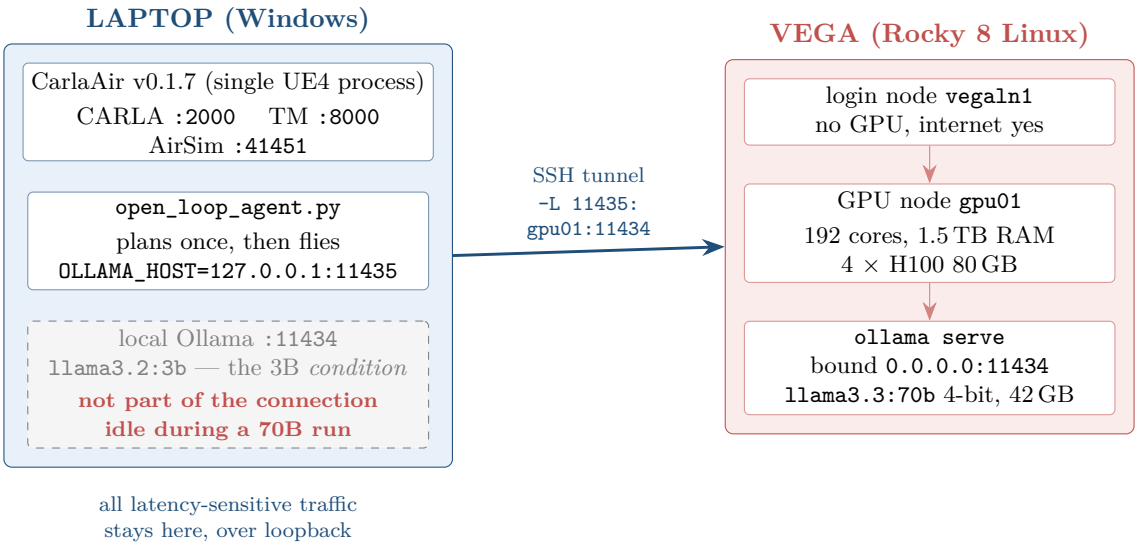


Figure 1: What runs where. Only the planning request crosses the network. The simulator, the agent's control loop, pose reads and the separation monitor all stay on the laptop over the loopback interface. **The greyed box is not infrastructure** — the laptop's own Ollama is a *condition being measured* (Section 2.3) and sits idle while the 70B is in use. It is drawn because it is there, and because it occupies port 11434, which is why the tunnel uses 11435.

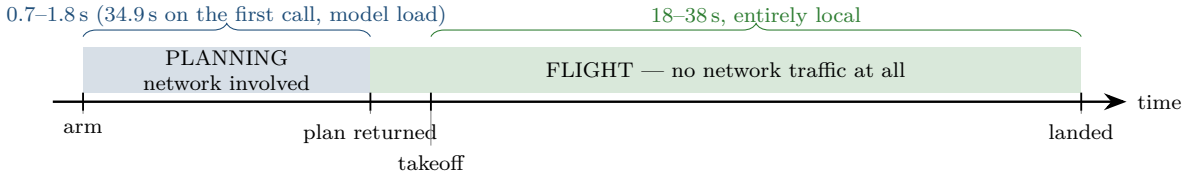


Figure 2: The open-loop timing argument. Because the single model call completes before takeoff, a slow or variable network cannot reach anything that is being measured about flight. Latency is recorded as a result in its own right, not as a confound.

2.3 Why a model also runs on the laptop

A reasonable reading of Figure 1 is that the laptop needs Ollama in order to reach Vega. **It does not.** Three copies of a model exist in this setup, for three different reasons, and only one of them is infrastructure.

Copy	Where	Why it exists
`llama3.2:3b`	**laptop**	the 3B *experimental condition*, run on realistic hardware
`llama3.2:3b`	Vega	proves the remote path does not change the model's output
`llama3.3:70b`	Vega	the large-model condition — the reason for all of this

The laptop's 3B is a thing being measured, not a helper. The research question is how planning quality changes with model capability, and 3B and 70B are two points on that curve. The upstream benchmarks cover only `llama3.1:8b` and `mistral-nemo`, so a 3B column is new — and if 3B holds up, it is the difference between this work running on hardware people already own and not.

That claim only means something if the 3B runs on the laptop. Run on an H100 it would measure a small model's planning quality while answering nothing about deployment. So it runs locally, on CPU, with `num_gpu` forced to `0` — because the laptop's 4 GB of VRAM is already spoken for by the simulator. That is the honest configuration for the question being asked, not a limitation worked around.

The 3B on Vega is a calibration instrument. Same weights, same prompt, `temperature 0`, two machines. If it produces plans identical to the laptop's, the whole remote path — tunnel, endpoint, option handling — is shown not to alter the model's behaviour. If it does not, that must be known *before* any 70B result is trusted. Two gigabytes of scratch buying confidence in forty-two.

The local model is also the hazard that set the port number

The laptop's Ollama occupies **11434**, which is Ollama's default. During a 70B session it is idle and purely a liability: had the tunnel used 11434 and then dropped, planning would have fallen through to the *local 3B* while the record still said 70B. Choosing 11435 turns that silent substitution into a refused connection. The system is safer precisely because we assumed the idle model might answer by accident — see Figure 6.

3 Vocabulary of high-performance computing

Every term used later, defined once. A reader who knows these can skip ahead.

3.1 The machine

Term	Meaning
Cluster	Many separate computers wired together and presented as one system. Vega is 44 computers.
Node	One of those computers. It has its own CPUs, memory and operating system.
Login node	The node you reach when you connect (`vegaln1`). For editing files, submitting work and moving data. **It has no GPU** and running computation on it is against policy, because every user shares it.
Compute node	Where work actually runs (`cn01`–`cn42`). You never log in directly; the scheduler gives you one.
GPU node	A compute node that also has graphics processors (`gpu01`, `gpu02`). These are the scarce resource: 8 GPUs serve the whole university.
Core	One independent processing unit inside a CPU. Vega's nodes have 192 each.
ppn	"processors per node" — how many cores you are asking for.
GPU	Graphics Processing Unit. Originally for rendering; now the standard hardware for neural networks because it performs thousands of multiplications simultaneously.
H100	NVIDIA's data-centre GPU. Vega's carry **80 GB** of on-board memory each.
VRAM	The memory physically on the GPU. A model must fit in it to run at full speed. This is the number that decides which models you can use.
Lustre	A parallel filesystem: storage spread across many servers so many nodes can read at once. `/scratch` lives on it.
InfiniBand	A very fast network between nodes, used instead of ordinary Ethernet.
Rocky 8.10	The Linux distribution Vega runs.
Zen4	The AMD CPU architecture of Vega's processors.

3.2 Getting work to run

Term	Meaning
Scheduler	The program that decides which job runs on which node and when. You never choose a node yourself.
Job	A request for resources plus the work to run on them.
PBS	Portable Batch System — the family of scheduler Vega uses. Its commands begin with `q`: `qsub`, `qstat`, `qdel`.
TORQUE	The specific PBS implementation here. **This matters:** its resource syntax is `nodes/ppn/gpus`. The PBS Pro syntax `select/ncpus/ngpus` is *accepted without error* and then never satisfied, so the job waits forever silently.
Moab	A second scheduler layered on top, which does the actual placement. Its commands are `showq` and `checkjob`, and they see *everyone's* jobs — `qstat` shows only your own.
Queue	A named waiting line with its own limits on walltime and size.
Walltime	Real elapsed time you are asking for. When it expires the job is killed, finished or not. Ask for enough; asking for far too much delays scheduling.
Interactive job	A job that gives you a shell on the allocated node instead of running a script (`qsub -I`). What this workflow uses.
OnDemand	Vega's web portal, which can launch interactive sessions through a browser.

3.3 Networking

Term	Meaning
VPN	Virtual Private Network. Makes your laptop appear to be on the university network. Vega refuses connections from anywhere else, and campus wireless counts as "anywhere else".
Duo / MFA	Multi-factor authentication — a second proof of identity, here a phone notification.
SSH	Secure Shell: an encrypted connection giving you a command line on a remote machine.
Port	A numbered channel on a machine, so one address can host many services. CARLA uses 2000, AirSim 41451, Ollama 11434.
Loopback	The address `127.0.0.1`, meaning "this same machine". Traffic never touches a network card.
Port forwarding	Making a port on one machine stand in for a port on another, through an SSH connection. The mechanism this whole manual depends on.
`0.0.0.0`	"listen on every network interface", as opposed to loopback only. A server bound to loopback cannot be reached by anyone else — including the login node.

3.4 Language models

Term	Meaning
Parameter	One learned number inside the model. "70b" means 70 billion of them. More parameters generally means more capability and certainly means more memory.

Term	Meaning
Quantisation	Storing each parameter in fewer bits. At 4 bits a 70-billion-parameter model occupies about 42 GB instead of roughly 140 GB at full precision. This is what makes it fit in one H100.
Ollama	A program that serves language models over a simple HTTP interface. Install it, point it at a model, make requests.
Token	A chunk of text, roughly a word-piece. Models read and write tokens.
Context length (`num_ctx`)	How many tokens the model can hold at once. **It costs memory**, and Section 11 shows how badly.
KV cache	Working memory the model keeps for the tokens currently in context. Its size grows with context length, and it is *separate* from the model weights.
`num_gpu`	In Ollama, how many of the model's layers to place on the GPU. `0` means pure CPU. Set it high on a cluster or a 42 GB model will run on processors and every timing will be meaningless.
Constrained decoding	Forcing the model's output to match a schema, so it must emit valid JSON rather than prose that happens to look like JSON.

4 Step 1 — Getting in

4.1 The network rules

Connecting from	VPN needed?
Daytona Beach campus, **wired**	no
Prescott campus, **wired**	no
Authorised residential campus networks	no
Campus **wireless**	**yes**
Anywhere off campus	**yes**

There is no exception permitting direct SSH from off campus.

4.2 The VPN, in order

1. Launch **Cisco AnyConnect Secure Mobility Client**.
2. Enter the gateway `dbvpn1.erau.edu` and click Connect.
3. Choose the user group: **Students** or **Faculty_&_Staff**.
4. Enter your ERAU username and password.
5. Complete **Duo**: `push` for a phone notification, `phone` for a call.
6. Wait until the client reads **Connected** before doing anything else.

4.2.1 Prove the route works before blaming SSH [verified]

```
Test-NetConnection vega.erau.edu -Port 22
```

`TcpTestSucceeded : True` means the path is open. Measured: `vega.erau.edu` resolves to **155.31.18.51**, with port 22 (SSH) and port 443 (the portal) both reachable once the VPN is up.

4.3 Logging in [verified]

```
ssh chikhala@vega.erau.edu
```

This lands you on the login node `vegaln1`. Confirm where you are and what shell you have:

```
hostname; echo $0; pwd; echo $HOME; echo $USER
```

The login node is not for computing

`vegaln1` is shared by every user on the system. Editing, submitting, monitoring and *downloading* are fine. Running a model there is both against policy and pointless, because **there is no GPU on a login node**.

5 Step 2 — Storage, and where the model goes

Two locations matter.

Location	Shorthand	Use
Home	`~` or `$HOME`	Small, backed up, quota-limited. Scripts and configuration.
Scratch	`/scratch/$USER`	Large, fast, on Lustre, **not backed up**. Model weights, datasets, anything bulky.

Check both before downloading 47 GB of models [verified]:

```
df -h ~ /scratch
du -sh /scratch/$USER
quota -s
lfs quota -h /scratch
```

Scratch is not a backup

Whether `/scratch` is automatically purged, and after how long, is an open question with the HPC team. Treat anything there as reproducible rather than precious, and keep the commands that recreate it — which is part of why this manual exists.

6 Step 3 — Installing Ollama without administrator rights

6.1 Why it has to be installed by hand

Constraint	Consequence
No `ollama`, `vllm` or Apptainer module exists	we must install it ourselves
No root access	`install.sh` cannot be used; it writes to `/usr`
Compute nodes have **no internet**	nothing can be downloaded from inside a job
The login node *does* have internet	stage everything from there, in advance

Ollama's Linux tarball is self-contained — it bundles its own CUDA libraries — so it unpacks into a user directory and needs no system changes. That is what lets an unprivileged user drive an H100.

6.2 The documented download URL is broken [verified]

A 9-byte "success"

`https://ollama.com/download/ollama-linux-amd64.tgz` returns **404**. It redirects to a release asset that no longer exists, because packaging moved from `.tgz` to `.tar.zst`. `curl` cheerfully reports "100%" having downloaded a nine-byte *Not Found* page. **Always check the file size.**

Find the real asset names first:

```
curl -s https://api.github.com/repos/ollama/ollama/releases/latest | grep
    ↪ browser_download_url
```

6.3 Installing [verified]

```
mkdir -p /scratch/$USER/ollama/models
cd /scratch/$USER/ollama

curl -L --progress-bar -o ollama-linux-amd64.tar.zst \
  https://github.com/ollama/ollama/releases/download/v0.35.1/ollama-linux-amd64.tar.zst

curl -sL -o sha256sum.txt \
  https://github.com/ollama/ollama/releases/download/v0.35.1/sha256sum.txt

sha256sum -c --ignore-missing sha256sum.txt
```

`-ignore-missing` is needed because that checksum file lists all 17 release assets and we have one of them.

Rocky 8's `tar` has no `-zstd` flag, so decompress through a pipe:

```
zstd -dc ollama-linux-amd64.tar.zst | tar -xf -
```

That yields `bin/` and `lib/`, 2.2 GB uncompressed. `lib/ollama` contains `cuda_v12` and `cuda_v13`, which is precisely why nothing needs to be installed system-wide. Delete the 1.4 GB archive afterwards; keep `sha256sum.txt` as a record of which release is installed.

This installs v0.35.1, which is no longer what Vega runs

The study moved to **v0.40.6** on 19 October 2026 (the runner records `ollama_version` on every row). To install fresh, substitute the current release tag in both URLs above. To move an existing install to a newer release, follow Section 8 instead, which is how 0.40.6 was installed.

6.4 The environment file [verified]

ERAU asks users not to put this in `.bashrc`, so it lives in a file you source deliberately:

```
cat > /scratch/$USER/ollama/env.sh <<'EOF'
export OLLAMA_HOME=/scratch/chikhala/ollama
export OLLAMA_MODELS=$OLLAMA_HOME/models
export PATH=$OLLAMA_HOME/bin:$PATH
export LD_LIBRARY_PATH=$OLLAMA_HOME/lib/ollama:$LD_LIBRARY_PATH
EOF
```

Every session then begins with:

```
source /scratch/$USER/ollama/env.sh
ollama --version
```

"could not connect to a running Ollama instance" is expected
That warning appears whenever no server is running. What matters is that the command prints a *client version* rather than a linker error — the latter would mean `LD_LIBRARY_PATH` is wrong.

7 Step 4 — Staging the models

`ollama pull` needs a running server, so start one briefly on the login node. This is a *download*, which ERAU lists as appropriate login-node work, not computation.

```
nohup ollama serve > /scratch/$USER/ollama/serve-login.log 2>&1 &
sleep 5; ollama list
```

An empty table *with headers* means the server is answering and `OLLAMA_MODELS` points at scratch.

```
ollama pull llama3.3:70b
ollama pull llama3.1:8b
ollama pull llama3.2:3b
ollama list
pkill -f "ollama serve"
```

Model	Size	Role in the study
`llama3.3:70b`	**42 GB**	the primary — the reason for all of this
`llama3.1:8b`	4.9 GB	the upstream benchmark's model; links our numbers to published ones
`llama3.2:3b`	2.0 GB	**control** — the identical model the laptop runs

47 GB in total.

Why the 2 GB control model earns its space
It is the same model the laptop runs. Same prompt, `temperature 0`, two machines. If Vega's 3B produces identical plans, the remote path is verified end to end. If it does not, that must be discovered *before* any 70B result is trusted.

Never run the 70B on the login node
Pulling is fine. `ollama run llama3.3:70b` on `vegaln1` would be CPU inference on a node shared by every user — against policy, and painfully slow.

8 Updating Ollama

Ollama has no self-updater for a tarball install. Updating means replacing `bin/` and `lib/` with a newer release's. The model store is separate (`OLLAMA_MODELS`) and is **not** touched, so the 47 GB of weights are not downloaded again.

The Ollama version is part of the apparatus

A plan is produced by the model *and* by the server running it. Two rules follow:

- **Never update in the middle of a block.** Every run in a block must come from the same server. The runner records `ollama_version` on every row, so a mixed block is visible afterwards, but it can't be undone.
- **Check plans before flying on a new version.** Run the mission set with `--plan-only` and compare the raw outputs with the last block. On 19 October 2026, the move from 0.35.1 to 0.40.6 changed nothing measured. But even on one version, a fresh server instance has been seen to change one plan (M09, 20 October), so a check costs a minute and settles it.

Note the change in the run ledger in the same commit as the first block flown on it.

8.1 What was done, 19 October 2026 [verified]

Vega went from 0.35.1 to **0.40.6**, on the login node `vegaln1` (compute nodes have no internet), with no server running. Each step below was run and its output observed. They are the same steps for any later release: substitute the version.

Step 1. See what you are replacing.

```
source /scratch/$USER/ollama/env.sh; ollama --version
```

```
Warning: could not connect to a running Ollama instance
Warning: client version is 0.35.1
```

The warning is expected with no server running. Also make sure nothing is serving from these files. This check was not part of the 19 October run, but replacing the binaries under a live server is the one way to break a running job:

```
pgrep -af "ollama serve"
```

It must print nothing.

Step 2. Pick a stable release. `releases/latest` returns the newest *stable* release, skipping pre-releases:

```
curl -s https://api.github.com/repos/ollama/ollama/releases/latest | grep '"tag_name"'
```

Don't use a release candidate

On 19 October the release list also showed `v0.40.7-rc0`, a pre-release, and the plain `v0.40.7` tarball returned 404. This server is the instrument the study's numbers come from, so it runs stable releases only. "We ran the benchmark on a pre-release" is a question nobody wants in a review.

Step 3. Download, about 1.44 GB. Saved under a versioned name, so the file says which release it is. Step 4 copes with that:

```
cd /scratch/$USER/ollama && curl -L --progress-bar -o ollama-0.40.6.tar.zst https://
    ↪ github.com/ollama/ollama/releases/download/v0.40.6/ollama-linux-amd64.tar.zst
```

Step 4. Check the size, then the hash. The size check catches the 9-byte 404 page from Step 3:

```
ls -lh ollama-0.40.6.tar.zst
```

```
curl -sL -o sha256sum-0.40.6.txt https://github.com/ollama/ollama/releases/download/v0
    ↪ .40.6/sha256sum.txt
grep -F "$(sha256sum ollama-0.40.6.tar.zst | cut -d' ' -f1)" sha256sum-0.40.6.txt &&
    ↪ echo VERIFIED
```

```
c164ab8b1b28e2a247c29719b8e404c423c5b83ae64c3ab4e59d50c7ff7fc5e0  ./ollama-linux-amd64.
    ↪ tar.zst
VERIFIED
```

That hash is v0.40.6's CUDA build. If the `grep` prints nothing, stop: nothing has been replaced yet.

Why `grep` and not `sha256sum -c`

On 19 October the archive was saved as `ollama-0.40.6.tar.zst`, and `sha256sum -c --ignore-missing` printed `no file was verified`. The download was fine. `-c` matches by *file name*, the checksum file only knows `ollama-linux-amd64.tar.zst`, and `--ignore-missing` turns "no match" into silence. Looking the hash up directly works whatever the file is called, and it can't pass by skipping.

Step 5. Replace the build. Only after `VERIFIED`:

```
rm -rf bin lib
zstd -dc ollama-0.40.6.tar.zst | tar -xf -
```

```
ollama-0.40.6.tar.zst: 2255124480 bytes
```

That is 2.25 GB, the same size as the old build.

Step 6. Confirm the version and the bundled CUDA.

```
ollama --version && ls lib/ollama | grep -i cuda
```

```
Warning: could not connect to a running Ollama instance
Warning: client version is 0.40.6
cuda_v12
cuda_v13
```

A *linker* error here instead means the extraction failed. The two `cuda_` folders are what let it drive an H100 with nothing installed system-wide.

Step 7. Confirm the models survived. They live in `models/`, which none of this touches.

```
du -sh models && ls models/manifests/registry.ollama.ai/library
```

That should show 47 GB and `llama3.1`, `llama3.2`, `llama3.3`.

`ollama list` needs a running server from 0.40.6 on

0.35.1 answered `ollama list` straight from the models directory. 0.40.6 asks the server, so on the login node with nothing running it can't list anything. Read the directory as above, rather than starting a server on the login node just to check. Inside a GPU job, with the server up, `ollama list` works as before.

Step 8. Tidy up. The archive is now dead weight on a filesystem that isn't backed up. Keep the checksum file as the record of what is installed:

```
rm -f ollama-0.40.6.tar.zst
```

8.2 Confirming in the next GPU job [verified]

Start the server as in Step 7. From the laptop, through the tunnel:

```
curl http://127.0.0.1:11435/api/version
```

```
{"version":"0.40.6"}
```

That is the version the runner records on every row. Then do the `--plan-only` check described at the top of this section before any flight.

8.3 Going back

No copy of 0.35.1 was kept: `rm -rf bin lib` removed it. To return to an older release, repeat Steps 3–6 with that release's tag. Its assets stay on GitHub. To make going back faster, move the old build aside in Step 5 instead of deleting it (`mv bin bin.0.35.1 && mv lib lib.0.35.1`), at a cost of 2.2 GB of `/scratch`.

9 Step 5 — Getting a GPU

9.1 What to ask for, and why 48 cores

On this cluster the number of cores you request determines how many GPUs you get, because 192 cores divided by 4 GPUs is 48 cores per GPU:

Cores	GPUs	VRAM	Supports
2–48	**1**	80 GB	**70B at 4-bit (42 GB) comfortably**; FP8 tightly
49–96	2	160 GB	70B at bf16 (≈140 GB)
97–144	3	240 GB	—
145–192	4	320 GB	405B at 4-bit (≈230 GB)

Cores do not make the model faster

At 4-bit the 70B sits entirely inside one 80 GB H100 and nothing spills to the CPU, so the GPU does essentially all the work. The extra cores are headroom for loading a 42 GB file off Lustre and for Ollama's own bookkeeping — not compute. Asking for more than 48 reserves H100s out of a pool of **eight for the whole university** that you will not use.

9.2 The queue situation, as actually found [verified]

Use `gpuq`. This is the current, working route:

```
qsub -I -q gpuq -l nodes=1:ppn=48:gpus=1,mem=64gb,walltime=04:00:00
```

On 9 October 2026 that started **immediately** — job **67630** on `gpu01`. Before the queue split, the same request waited around two hours because CPU-only jobs were occupying the GPU nodes.

This reversed between 6 and 9 October, and the lesson generalises

On **6 October** `gpuq` did not exist — submitting to it returned `Unknown queue MSG=requested queue not found` — while the *restriction* half of the change was already live, with `checkjob` showing `[Class: 2]` rejecting the GPU nodes for `normalq`. The only way onto a GPU was `batch`, which the restriction did not cover.

On **9 October** that inverted: `gpuq` is live, and `batch` is **disabled** — `qsub` returns `Queue is not enabled MSG=queue is disabled`. A command that worked on Monday failed on Thursday.

So check before assuming. `qstat -Q` costs nothing and tells you which queues exist and which are enabled:

```
qstat -Q
```

Read the `Ena` column — `yes` means the queue accepts submissions. On 9 October:

```
Queue              Max    Tot   Ena   Str   Que   Run   Hld
batch                0      1    no    no     0     1     0
normalq              0      7   yes   yes     4     3     0
longq                0     31   yes   yes     2    29     0
shortq               0      0   yes   yes     0     0     0
gpuq                 0      3   yes   yes     1     2     0
```

`batch` shows `Ena no` with one job still running: disabled, and draining. `gpuq` shows three jobs, two running and one queued.

9.3 Reading the request

Fragment	Meaning
`qsub`	submit a job
`-I`	interactive — give me a shell on the node rather than run a script
`-q gpuq`	put it in the `gpuq` queue, the one for GPU work
`-l`	the resource list follows
`nodes=1`	one machine
`ppn=48`	48 cores on it, which maps to one GPU
`gpus=1`	one GPU
`mem=64gb`	system RAM, distinct from the GPU's 80 GB
`walltime=04:00:00`	four hours, after which the job is killed

9.4 Waiting, and finding out why [verified]

```
showq                  # EVERYONE's jobs, with time remaining
showq -i               # idle/eligible jobs, in priority order
showq -b               # blocked jobs: over a policy limit, not waiting on hardware
checkjob 67322         # why is THIS job not starting
qstat -u $USER         # my jobs only
qstat -Q               # what queues exist
qdel 67322             # cancel
```

Job states: `Q` queued, `R` running, `C` complete, `H` held, `E` exiting.

`qstat` cannot see other users

On Vega `qstat -a` shows only your own jobs, which makes the cluster look empty when it is full. Use **`showq`** for the real picture. "All nodes and GPUs appear free" in the portal is not evidence that a GPU is available to you.

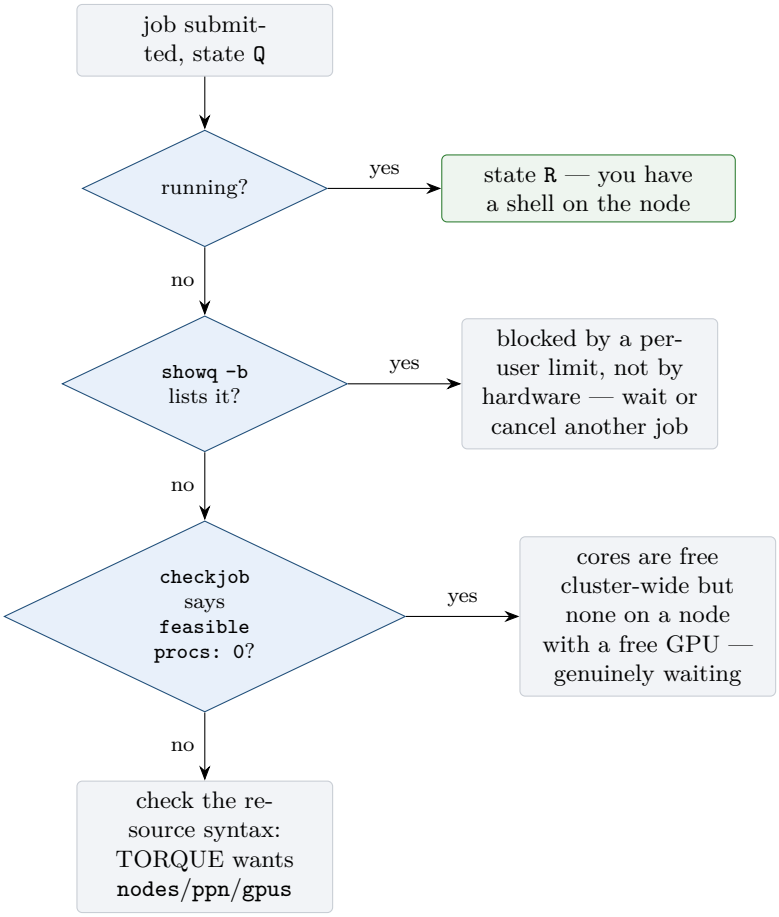


Figure 3: Diagnosing a job that will not start. `checkjob` is the single most useful command on the cluster; guessing is the least useful.

9.5 Nodes are shared, so a modest request gets in sooner [verified]

Jobs with 1, 8, 12, 32 and 192 cores run side by side on the same node. The `jobs = 0-191` seen in `pbsnodes` output is one job that *asked for* all 192, not a policy of whole-node allocation. So a small request can slot into the gap a large one leaves: measured on 5 October 2026, a 48-core single-GPU job was feasible on a node where the three higher-priority jobs ahead of it consumed only 140 of 192 cores.

9.6 Before you ask: which node has an idle card? [verified]

Vega has two GPU nodes, `gpu01` and `gpu02`, with four H100s each. Before submitting, look at both, so you can ask for the node that actually has a free card. From the login node, SSH straight into each one. This works without holding a job on the node:

```
ssh gpu01 nvidia-smi --query-gpu=index,memory.used,utilization.gpu --format=csv
ssh gpu02 nvidia-smi --query-gpu=index,memory.used,utilization.gpu --format=csv
```

A card is idle when it shows near-zero memory *and* 0% utilisation, like card 2 here:

```
index, memory.used [MiB], utilization.gpu [%]
0, 18697 MiB, 42 %
1, 17675 MiB, 41 %
2, 0 MiB, 0 %
3, 17585 MiB, 36 %
```

Memory matters more than utilisation. A card with a model loaded and waiting for requests shows 0% but tens of GB in use, and it belongs to someone. To see *what* is on a busy card:

```
ssh gpu01 nvidia-smi --query-compute-apps=gpu_uuid,pid,process_name,used_memory --
    ↪ format=csv
ssh gpu02 nvidia-smi --query-compute-apps=gpu_uuid,pid,process_name,used_memory --
    ↪ format=csv
```

`nvidia-smi` shows what the cards are doing *now*. A GPU the scheduler has just handed to another job can still look idle until that job loads something, so if a request on a node with an idle-looking card doesn't start, `checkjob` (Figure 3) is the next step.

9.7 Asking for a specific node, and pointing the tunnel at it

The plain request (`nodes=1`) lets the scheduler choose. To land on the node with the idle card, name it:

```
qsub -I -q gpuq -l nodes=gpu02:ppn=48:gpus=1,mem=64gb,walltime=04:00:00
```

Naming a node trades flexibility for certainty. If that node can't host the job, it waits even though the other node is free. Check first.

The tunnel must name the node the job is on

Whichever node you get, the prompt says so (`[chikhala@gpu02 ~]$`) and so does `hostname`. The tunnel in Section 12 names a node explicitly, so a job on `gpu02` needs `-L 11435:gpu02:11434`. A tunnel still pointing at `gpu01` reaches whatever happens to be listening there: nothing, or someone else's server.

Every new job also needs its own `ollama serve`. The server dies with the job that started it. Seen on 20 October 2026: a new job on `gpu01`, the old tunnel still up, and `curl` returning `Recv failure: Connection was reset` until the server was started inside the new job.

10 Step 6 — Which GPU is mine?

Inside the job, this is the first thing to settle, and the documented mechanism did not work.

Both allocation variables were empty [verified]

On the allocated node, `CUDA_VISIBLE_DEVICES` and `PBS_GPUFILE` were both **empty**, so nothing told us which of the four cards was ours. GPUs appear not to be isolated per job, which means a careless user can disturb someone else's work.

A node is not a card, and the tunnel wants the node

`gpu01` is a **machine** — the cluster has two, `gpu01` and `gpu02`. The `2` in `CUDA_VISIBLE_DEVICES=2` is a **card inside that machine**, which has four numbered 0–3. They are unrelated numbers that both happen to be small.

It matters because the tunnel in Section 12 names a *host and port*. Cards are not network-addressable: the server listens on one port on `gpu01` whichever card it uses. So the tunnel says `gpu01` regardless — and if the scheduler had given you `gpu02`, pointing the tunnel at `gpu01` would reach a server someone else is running.

The workaround, which is reading rather than guessing where possible:

```
nvidia-smi
nvidia-smi -L
nvidia-smi --query-gpu=index,name,memory.total,memory.used,utilization.gpu --format=csv
nvidia-smi --query-compute-apps=pid,process_name,used_memory --format=csv
```

Find a card reporting near-zero memory in use, then pin to it explicitly so nothing strays onto a neighbour's GPU:

```
export CUDA_VISIBLE_DEVICES=0
```

This is a courtesy issue with teeth

Two of `gpu01`'s four cards were idle while another job held the node. Pinning kept our 42 GB model off the two that were busy. A user who does not check could silently halve someone else's throughput — which is why this is question one in the outstanding list for the HPC team.

11 Step 7 — Serving the model, and the memory trap

11.1 Starting the server [verified]

One command at a time, because each one answers a different question.

Load the environment — this is what puts `ollama` on your `PATH` and points it at the models in `/scratch`:

```
source /scratch/$USER/ollama/env.sh
```

Confirm it loaded. A *client version* is success:

```
ollama --version
```

"could not connect to a running Ollama instance" is expected here

No server is running yet, so the client cannot reach one. What matters is that it prints a version rather than a linker error — a linker error means `LD_LIBRARY_PATH` is wrong and the bundled CUDA libraries were not found.

See the four cards and pick an idle one:

```
nvidia-smi --query-gpu=index,memory.used,utilization.gpu --format=csv
```

A real reading from 9 October 2026 — three cards were busy with somebody else's job and only one was free:

```
index, memory.used [MiB], utilization.gpu [%]
0, 18697 MiB, 42 %
1, 17675 MiB, 41 %
2, 0 MiB, 0 %
3, 17585 MiB, 36 %
```

Pin to the free one. Had we defaulted to 0, our 42 GB would have landed on top of a running job:

```
export CUDA_VISIBLE_DEVICES=2
```

Start the server, which is the one command worth reading in full:

```
OLLAMA_HOST=0.0.0.0:11434 OLLAMA_CONTEXT_LENGTH=4096 nohup ollama serve > /scratch/
    ↪ $USER/ollama/serve-gpu.log 2>&1 &
```

Fragment	What it does
`OLLAMA_HOST=0.0.0.0:11434`	Binds the server to **every** network interface rather than loopback only. This is the piece that makes the tunnel possible: bound to `127.0.0.1` the server is invisible to the login node and the forward is refused. Set **inline** rather than exported, so it applies to this one process. Your own `ollama` client commands afterwards still default to `127.0.0.1` and reach it locally.
`OLLAMA_CONTEXT_LENGTH=4096`	Caps the context window. The default is **131,072**, which needs about 43 GB of KV cache on top of 42 GB of weights — 85 GB against an 80 GB card. See Figure 4.
`nohup`	"No hang up". Detaches the process from this terminal's hangup signal, so it survives if your SSH session drops.
`ollama serve`	The server itself.
`> .../serve-gpu.log`	Sends normal output to a log file. `$USER` expands to your username.
`2>&1`	Sends **error** output (stream 2) to the same place as normal output (stream 1), so a failure is captured rather than lost.
`&`	Runs it in the background and returns your prompt.

It inherits `CUDA_VISIBLE_DEVICES` from the previous command, so it uses only the card you pinned. You get a job number such as `[1] 3416817` back — that is the process id.

Check the server is answering and can see the staged models:

```
ollama list
```

Load the model. The first call reads 42 GB off Lustre, so expect roughly 35–45 seconds before it replies:

```
ollama run llama3.3:70b "reply with the single word: ready"
```

And the check that actually matters:

```
nvidia-smi --query-gpu=index,memory.used,utilization.gpu --format=csv
```

Your card should read about **42,000 MiB**. A utilisation of 0 % is correct for an idle server between requests — memory is the measurement here, not utilisation.

11.2 Finding the server again later

Command	Answers
`jobs -l`	background jobs *of this shell*, with their process ids
`pgrep -af ollama`	every `ollama` process on the node, with its full command line, whoever owns it
`nvidia-smi -query-compute-apps=pid,used_memory -format=csv`	which process ids are actually **on a GPU**, and how much each is using

That last one is the one worth knowing: it ties a process to GPU memory, so it tells you whether *your* server is the thing holding the 42 GB rather than leaving you to assume. To stop the server: `kill <pid>`, or `pkill -f "ollama serve"`.

Binding `0.0.0.0` rather than loopback is what allows the login node to reach the server, which is what makes the tunnel possible. Setting it inline rather than exporting keeps your own `ollama` client commands pointed at `127.0.0.1`.

The last `nvidia-smi` is the measurement that matters

Roughly **42 GB of VRAM in use** means the model is on the GPU. Near zero means it quietly fell back to CPU and every timing you collect will be meaningless.

11.3 Why the context length nearly ruined the experiment [verified]

This is the subtlest problem in the whole setup and worth understanding properly.

A model needs memory for two separate things: its **weights**, which are fixed, and its **KV cache**, which scales with how many tokens it may hold at once. Ollama's default context length for these models is **131,072** tokens. On a 70B model that cache is about 43 GB.

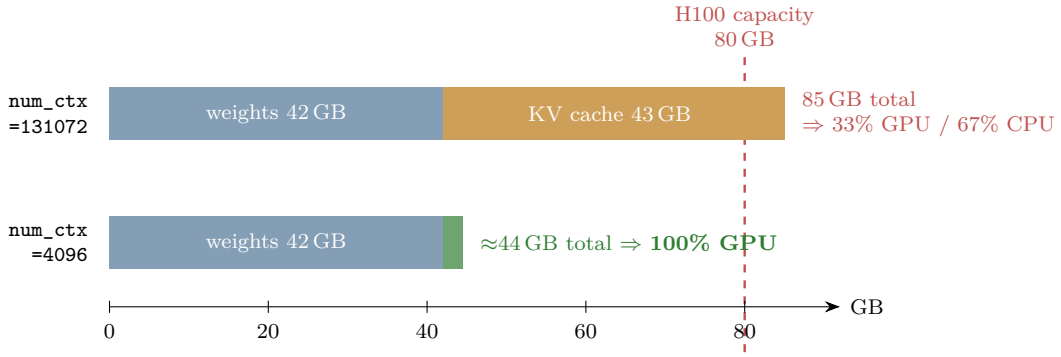


Figure 4: The context-length trap. Leaving `num_ctx` at its default made the model 5 GB too large for the card, and Ollama responded by splitting it 67% onto the CPU — which is both far slower and no longer the measurement we intended. Our planning prompts are a few hundred tokens, so 4096 is ample and keeps the model entirely on the GPU.

Reducing context is not a compromise here

The agent's prompts are a few hundred tokens and its replies are capped at 512. A 4096-token window is many times what the task needs. The number to record in the protocol is the one that was used, which is why `num_ctx` is a declared field in the experiment's configuration rather than a default left implicit.

12 Step 8 — The SSH tunnel

12.1 What a tunnel actually is

An SSH tunnel makes a port on your laptop behave as though it were a port on a remote machine. Traffic sent to the local port travels inside the encrypted SSH connection, emerges at the other end, and is forwarded onward.

The important subtlety for this workflow: the forwarding happens *from the login node*. You do not need SSH access into the compute node — you only need the login node to be able to open an ordinary TCP connection to it, which it can. This is what makes the whole arrangement possible without special privileges.

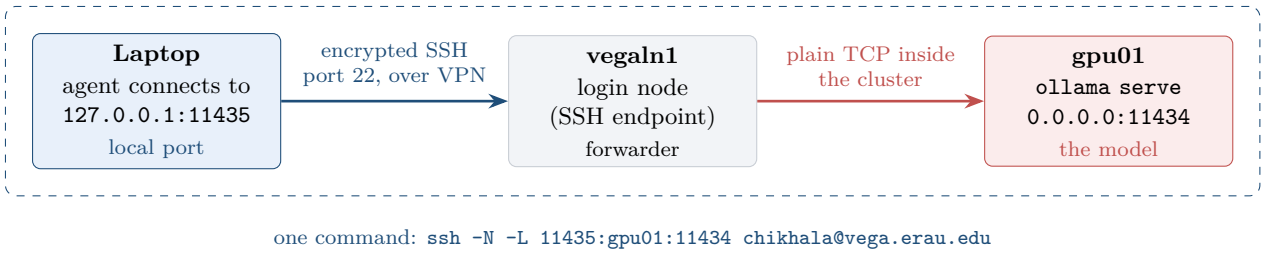


Figure 5: The three-hop path. The laptop never speaks to `gpu01` directly; the login node relays. Note that `gpu01` must be bound to `0.0.0.0` for the middle hop to be permitted at all.

12.2 Opening it [verified]

From the laptop, with the VPN connected, substituting the node name you noted from `hostname`:

```
ssh -N -o ServerAliveInterval=30 -L 11435:gpu01:11434 chikhala@vega.erau.edu
```

Fragment	Meaning
`-N`	do not run a command — forward ports only. The terminal will appear to hang; that is correct, and it must stay open.
`-o ServerAliveInterval=30`	send a keepalive every 30 s so an idle tunnel is not dropped by a firewall
`-L 11435:gpu01:11434`	forward *my* 11435 to `gpu01`'s 11434, resolved *from the login node*

Then point the agent at it:

```
$env:OLLAMA_HOST = "http://127.0.0.1:11435"
```

12.3 Why 11435 and never 11434 — a data-integrity decision

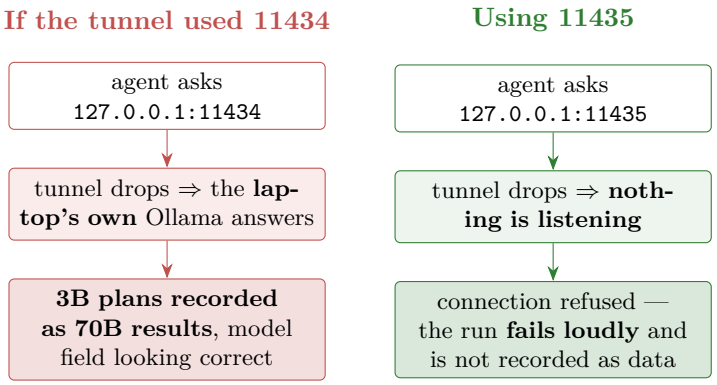


Figure 6: The port choice is not cosmetic. The laptop already runs Ollama on 11434 with `llama3.2:3b` as the control model. Reusing that port would make a dropped tunnel *silently* substitute the small model for the large one, producing plausible, wrong, unrecoverable data. A distinct port converts that into an immediate, obvious failure.

This project has already lost one dataset to a measurement that looked fine and was not. Choosing a port that fails loudly is cheaper than discovering the substitution afterwards — which, in general, cannot be done at all.

13 Step 9 — Wiring the agent, and refusing to fly blind

A tunnel that works is not sufficient. The record has to *prove* which model produced each plan, so three changes were made to the agent.

13.1 The endpoint is explicit and recorded

```
OLLAMA_ENDPOINT = os.environ.get("OLLAMA_HOST") or "http://127.0.0.1:11434"
```

It is accepted either as `host:port` or as a full URL, normalised once, and written into every planning record and the provenance manifest. A result that cannot say where it came from is not evidence.

13.2 A pre-flight check that refuses to proceed

Before any aircraft is armed, the agent asks the endpoint which models it serves and requires an exact match for the one the protocol declares:

```
curl -s http://127.0.0.1:11435/api/tags
```

If the endpoint is unreachable, or reachable but serving a different model, the run is refused with a message that mentions the tunnel. The alternative — 30 missions flown against the wrong model — is a wasted session at best.

13.3 Transport failures are not model failures

A dropped VPN must not be recorded as the model producing an invalid plan. Exceptions are classified by origin, so a network failure is marked `transport_error` and **excluded from both the numerator and the denominator** of the correctness score. Otherwise a flaky connection would look like a less capable model.

13.4 Inference options are declared, not defaulted

Option	Value	Why it is recorded
`num_gpu`	999	all layers on the GPU. The laptop forces `0` because its VRAM belongs to the simulator; the data must say which
`num_ctx`	4096	see Figure 4 — the default would move 67% of the model to the CPU
`temperature`	0	deterministic output, so repeats are repeats
`num_predict`	512	a plan is short; this bounds a runaway reply

14 Step 10 — Proving it works, in order

Each rung tests one thing, so a failure localises itself.

#	Check	Passes when
1	`Test-NetConnection vega.erau.edu -Port 22`	`TcpTestSucceeded : True`
2	`ssh chikhala@vega.erau.edu`	a prompt on `vegaln1`
3	`qstat -u $USER` inside the job	state `R`
4	`nvidia-smi` on the node	a card with free memory
5	`nvidia-smi` after loading	≈**42 GB** in use
6	`curl 127.0.0.1:11435/api/tags` from the laptop	JSON listing `llama3.3:70b`
7	the agent's pre-flight check	"endpoint serving `llama3.3:70b`"
8	one mission, `-plan-only`	a valid plan, no simulator needed
9	one mission, flown	plan correct *and* the aircraft lands
10	the control model	Vega's 3B matches the laptop's 3B plan for plan

14.1 What the working system looked like [verified]

Model	`llama3.3:70b`, 4-bit, 42 GB
Placement	**100% GPU** on one H100 80 GB
Host	Vega, node `gpu01`, 4 h walltime. Job **67322** via `batch` on 6 October; job **67630** via `gpuq` on 9 October, after the queue split — see Section 9.2
Reached by	SSH tunnel through `vegaln1`, laptop port 11435
Simulator	local — CarlaAir v0.1.7, Town10HD, Epic quality, traffic on
Round trip	≈3.7 s for a cold `/api/tags`; 0.1 s once warm
Planning latency	**34.9 s** on the first call (loading 42 GB), then **0.7–1.8 s**
Delivered	30 missions, 42 planning calls, every one answered

The result worth knowing

`llama3.2:3b` on this laptop's CPU plans in 1.2–1.3 s. The 70B on an H100 plans in **0.7–1.8 s** — roughly *twice as fast* despite being twenty-three times larger, because the work moved off laptop cores that the simulator also needs. In this architecture there is no latency penalty for the larger model.

15 Step 11 — Running the experiment, from the laptop

Everything so far was setup. This is the command that produces data.

15.1 Three things the laptop needs first

15.1.1 The conda environment

```
conda activate carlaAir
```

A fresh terminal does NOT have it

Opening a new PowerShell gives you Miniconda's **base** interpreter, which has no `airsim` and no `ollama` package, and the run dies on `ModuleNotFoundError: No module named 'airsim'`. Nothing is activated by default — `$env:CONDA_DEFAULT_ENV` is empty in a new window. You can tell it worked because the prompt gains a `(carlaAir)` prefix.

To skip activation entirely, name the interpreter directly — this works in any window:

```
& "<miniconda>\envs\carlaAir\python.exe" scripts\run_missions.py ...
```

15.1.2 The simulator

```
cd $env:CARLAAIR_ROOT
.\CarlaAir.ps1 Town10HD
```

`$env:CARLAAIR_ROOT` is PowerShell's syntax for reading an environment variable, set once in `SETUP.md` so no document has to name one machine's drive letter. If it is not set, `cd` silently does nothing — check with `echo $env:CARLAAIR_ROOT` before blaming anything else.

15.1.3 The tunnel

Open, and left open, in its own window. Section 12.

15.2 The command

```
cd "<repository root>"; python scripts\run_missions.py --missions M01,M02,M03,M04,M05,
    ↪ M06,M07,M08 --block v4-polled --ollama-host http://127.0.0.1:11435
```

Fragment	What it does
`cd "..."`	The working directory **matters**. The runner resolves the mission config, the results folder and the agent relative to the repository, so running it from elsewhere finds the wrong files or none. Quoted because the repository folder's name contains a space.
`;`	PowerShell's statement separator — run the second command after the first, regardless of whether it succeeded. (PowerShell 5.1 has no `&&`.)
`python`	Must be the `carlaAir` interpreter. See above.
`scripts\run_missions.py`	The experiment entry point. `scripts/` produces evidence; `tools/` never appears in a result.
`--missions M01,...,M08`	Comma-separated mission ids, matching the `[[mission]]` entries in `configs/missions/baseline_v1.toml`. Omit it and every mission in the config runs.
`--block v4-polled`	A short label for this session. It appears in every filename written, so the results folder reads as a list of named sessions rather than bare timestamps. Purely organisational — it changes nothing about the flight.
`--ollama-host http://127.0.0.1:11435`	**The endpoint.** Port 11435 is the local end of the SSH tunnel, so this is what sends planning to the 70B on Vega. It is recorded in every planning record and in the provenance manifest, so the data can say where each plan came from.

15.2.1 Flags not used here, and when they matter

`--plan-only`	Plan and score **without flying**. Needs no simulator at all, so it is the fast way to check the model end of the pipeline.
`--repeats N`	Override the protocol's repeat count. The protocol says 3, which is where "30 runs" comes from: 10 missions × 3.
`--model NAME`	Override the config's model, e.g. to run the `llama3.2:3b` control against the same set.
`--option KEY=VALUE`	Override one inference option, repeatable — e.g. `--option num_gpu=999`. Whatever is used is recorded. The laptop needs `num_gpu=0` because the simulator owns its VRAM; a GPU server does not.
`--pause`	Wait for a keypress between runs, for watching each one.
`--config` / `--out`	Point at a different mission set or results folder. Rarely needed.

15.3 What it then does, in order

1. Reads the mission config and the `[protocol]` block — model, repeats, map, quality, traffic, inference options.
2. **Checks the endpoint before anything else**: asks what models it serves and requires an exact match for the one the protocol declares. A dead tunnel stops the run here rather than 30 missions later.
3. Writes `settings.json` for the roster the missions need, and snapshots the code that is about to fly into the results folder.
4. For each mission, for each repeat: resets the world, asks the model for one plan per drone, validates each against the schema, then flies them concurrently through `fly_plans` with the separation and collision monitor running.
5. Writes a JSONL file — one line per run, holding the plan, the raw model output, the latency, the flight outcome, the altitude reached after every step, and the provenance — plus a Markdown summary table.

Where the output goes, and why there are two places

`phases/phase-01-baseline-freeze/results/` holds the curated, tracked evidence for one scored series. `runs/agent-log.jsonl` is a separate, git-ignored log of *every* planning call ever made on the machine, whatever invoked it. The first is the experimental record; the second is a black box recorder. Neither replaces the other.

15.4 Expected timings

First planning call	**35–47 s** — loading 42 GB off Lustre, once per session
Every call after	**0.7–2.8 s**, depending on plan length
One single-drone run	about a minute, planning and flight together
24 runs (M01–M08)	roughly 15 minutes

Run the multi-drone missions as their own block

The runner spawns the whole roster at the start of a block. Run all ten missions together and the single-drone missions fly with four drones sitting on the pad — different conditions from a 24-run block, and not comparable with it. Fly `M01-M08`, then `M09`, then `M10`, as three commands.

16 Step 12 — The interactive session

16.1 The idea

A benchmark answers the questions you thought of in advance. Ten missions, three repeats, a score. What it cannot tell you is what happens at the edges of the plan language — what the model does with a phrasing nobody anticipated, or an instruction the eight available actions cannot express.

```
python tools\fly_chat.py
```

So this is a **sandbox**: ask for any number of drones at any speed, type an instruction in plain English, look at the plan the model produced, and decide whether to fly it. Nothing is scored and nothing is recorded.

Why it has to share the study's code path

A tool that flew missions *differently* from the runner would teach you things that are not true of the experiment. So `fly_chat.py` does not implement flight: it calls `plan_for` and `fly_plans`, the same functions the runner uses, and inherits the same system prompt, schema, validator, inference options, thread model, ground barrier and separation monitor. The agent's own comment on `fly_plans` says this outright — shared *on purpose*, because otherwise "scored runs and hand-flown runs would not be the same experiment, and nobody would notice until the numbers disagreed".

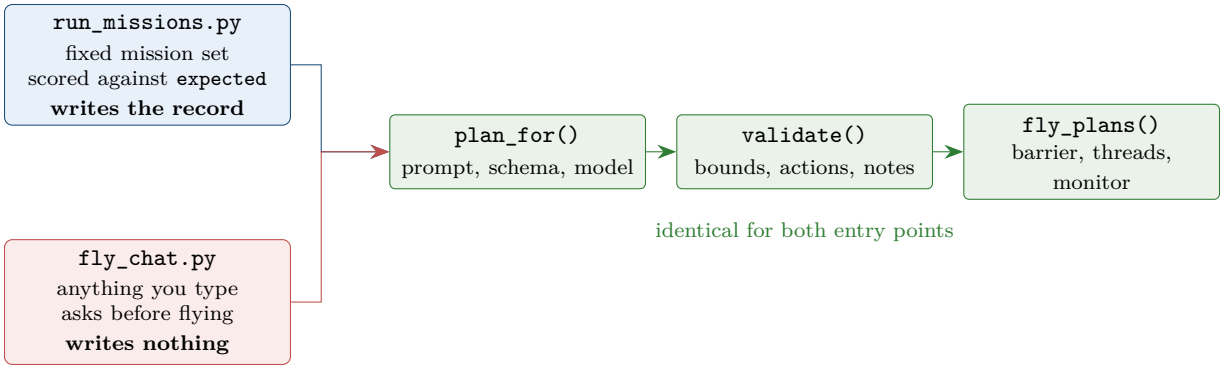


Figure 7: Two entry points, one flight path. The differences are confined to what goes in (a fixed set versus whatever you type) and what comes out (a scored record versus nothing at all). Everything between them is shared, which is what makes an observation in the sandbox worth anything.

16.2 What a session looks like

Figure 8 is the whole loop. Every path returns to the prompt, and the only route to the aircraft moving is an explicit `y`.

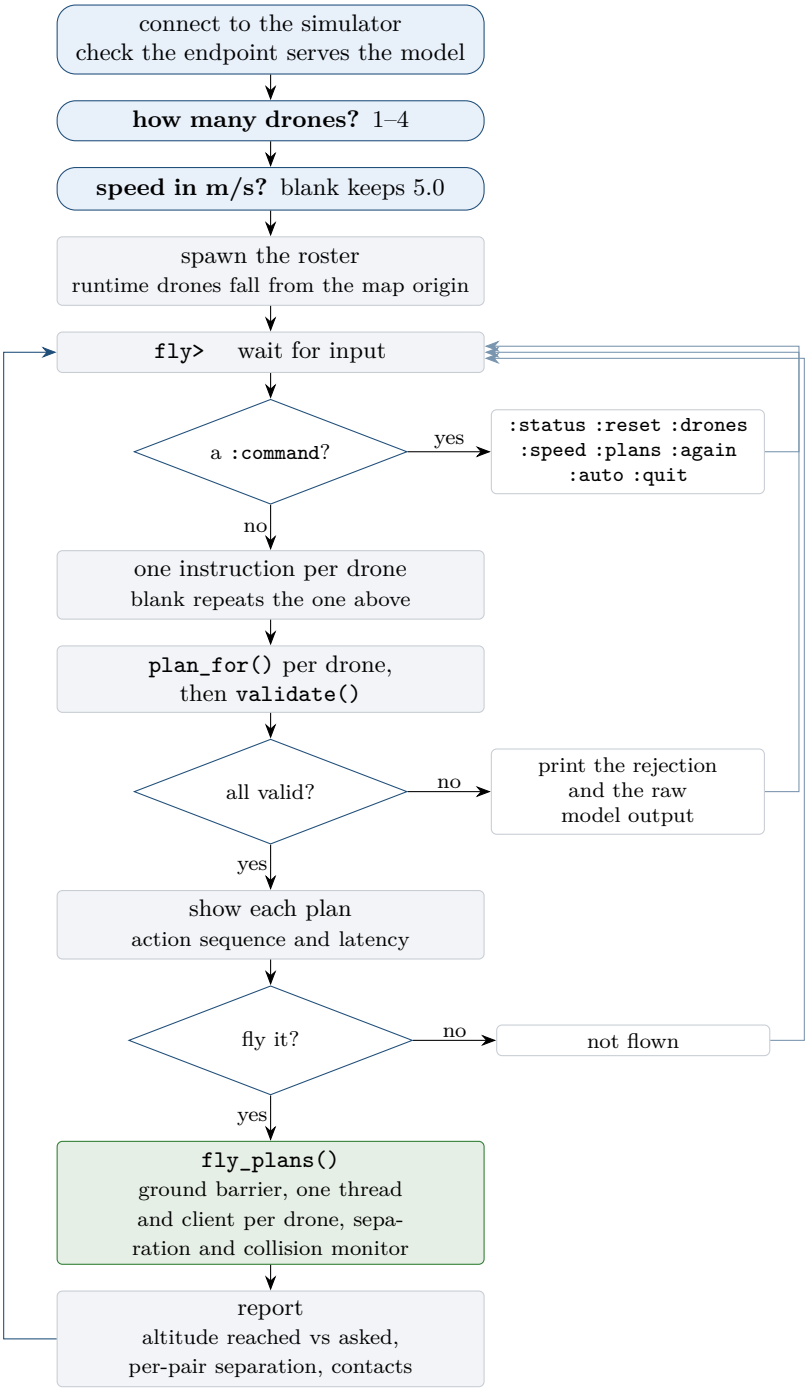


Figure 8: The session loop. Every path returns to the prompt, and the only route to the aircraft moving is an explicit `y`. A rejected plan prints the raw model output rather than failing silently, because what the model emitted is usually the interesting part.

16.3 The details that matter

Drone count	Asked once at the start, changeable with `:drones N`. More than one spawns them into the running simulator immediately, not at flight time.
Speed	Asked once, changeable with `:speed N`. It also **rewrites the system prompt**, because the speed is baked into it when the module loads — without that the aircraft flies at one speed while the model reasons about another, and every duration it chooses is wrong.
Confirmation	On by default. `:auto` turns it off.
Recording	None. `plan_for` and `fly_plans` do not log; only the runner's `main()` does. So no results file and no entry in `runs/agent-log.jsonl`.
Recovery	`:reset` returns every drone to its start pose. That is the exit from a stranded or wedged aircraft.

What it cannot do, and why that is the agent's limit not the tool's

Eight actions and no yaw, so no circling, orbiting, following or searching — and "turn left" can only become a sideways strafe. No camera and no sensing: the model plans before take-off and is never consulted again, so "find the red car" or "stop at the wall" are outside the architecture, not merely the vocabulary. The drones are not told about each other, so "follow Drone1" cannot work. And there is **no collision avoidance anywhere in the stack**. All of that is the frozen Phase 1 baseline; the sandbox simply lets you watch it honestly.

`:status`	where every drone is
`:reset`	return all drones to their start poses — the recovery
`:drones N`	change the roster mid-session
`:speed N`	change speed, *and* tell the model about it
`:plans`	the raw JSON the model returned
`:again`	re-fly the last plans without asking the model
`:auto`	stop asking for confirmation

16.4 Two findings it produced on its first use

Both from one instruction — *fly in circles clockwise with a diameter of 30 metres, three times, then land where you took off.*

The model treats `fly_right` as a turn. It is a strafe. The 70B built the circle as `fly_straight` → `fly_right`, repeated — which closes a loop only if the aircraft's heading changes. Nothing in this agent yaws, so forward is always the same world direction and the plan flew a diagonal staircase about 225 m out. The model's reasoning was sound for the action *names* it was given; the names do not match the semantics.

An unbounded `fly_to` is a hard hang. The return leg met a hotel. `moveToPositionAsync` had no timeout, so the drone pushed into the building indefinitely, holding its worker thread, which held the main thread — and `Ctrl-C` could not be delivered into a blocked RPC call. The only exit was killing the simulator. Now bounded, the same way `land` is.

There is no collision avoidance anywhere in this stack

Vertical staggering and spawn spacing are the only separation there is. A plan that flies 200 m across a city will meet a building, and the aircraft will not avoid it. Fly above the skyline, or slow down with `:speed 1` so every distance shrinks fivefold.

17 Troubleshooting

Symptom	Cause and remedy
`Unknown queue MSG=requested queue not found`	That queue does not exist. `qstat -Q` lists the real ones.
`Queue is not enabled MSG=queue is disabled`	The queue exists but is closed to new jobs. `batch` went this way on 9 October when `gpuq` went live. Check the `Ena` column of `qstat -Q`.
`ModuleNotFoundError: No module named 'airsim'`	The conda environment is not active. `conda activate carlaAir`, or name the interpreter in full.
`cd $env:CARLAAIR_ROOT` does nothing	The variable is not set in *this* window. `setx` only affects new sessions. Check with `echo $env:CARLAAIR_ROOT`.
A drone wedges against a building and the run will not stop	Contact with anything can stall a flight call. `Ctrl-C` cannot interrupt a blocked RPC call, so kill the simulator: `Stop-Process -Name CarlaUE4,CarlaUE4-Win64-Shipping -Force`.
Job queues forever, no error	Almost always TORQUE syntax. Use `nodes/ppn/gpus`; `select/ncpus/ngpus` are accepted and never satisfied. Confirm with `checkjob`.
`checkjob` says `feasible procs: 0`	Cores are free cluster-wide but none on a node with an available GPU. Genuinely waiting, not misconfigured.
Portal GPU session never starts	The portal submits GPU sessions to `normalq`, which is being restricted to CPU nodes, making the request unsatisfiable *with no error shown*. Submit from the terminal instead.
Cluster looks empty but nothing runs	`qstat` shows only your jobs. Use `showq`.
`nvidia-smi` shows near-zero memory	The model fell back to CPU. Check `num_gpu`, and check `num_ctx` (Figure 4).
Model loads at 33% GPU / 67% CPU	The KV cache made it exceed 80 GB. Set `OLLAMA_CONTEXT_LENGTH=4096`.
`curl` on 11435 refuses	The tunnel is down, or the server was bound to loopback instead of `0.0.0.0`, or the job's walltime expired.
Tunnel listening, nothing answers	The job ended. The forwarder survives the job, so `netstat` still shows a listener — check `qstat -u $USER`.
`curl: (56) Recv failure: Connection was reset` on 11435	The tunnel and SSH hop work, but nothing is listening on `<node>:11434`. Either the new job has no `ollama serve` yet (start it, Step 7), or the job is on the other node and the tunnel still names the old one (Section 9.6). Seen 20 October 2026.
`ollama -version` gives a linker error after an update	The new `lib/` did not extract cleanly. Extract again, or reinstall a release by its tag (Section 8).
`sha256sum: ... no file was verified`	The archive was saved under a different name from the one in the checksum file. Look the hash up with `grep` instead (Section 8, Step 4). Seen 19 October 2026.
`ollama list` fails on the login node	From 0.40.6 it needs a running server. Read `models/manifests/registry.ollama.ai/library` instead.
Downloaded archive is 9 bytes	The documented Ollama URL 404s. Use the `.tar.zst` asset and check the size.
`tar: unrecognized option -zstd`	Rocky 8's tar is too old. Pipe: `zstd -dc file.tar.zst | tar -xf -`
`could not connect to a running Ollama instance`	Expected with no server running. A *linker* error instead means `LD_LIBRARY_PATH` is wrong.
`CUDA_VISIBLE_DEVICES` empty	Not populated on this cluster. Read `nvidia-smi`, pick an idle card, pin it explicitly.

18 Shutting down properly

Eight GPUs serve the whole university, so holding an idle one is a policy matter rather than a courtesy.

```
pkill -f "ollama serve"     # on the GPU node
exit                        # ends the interactive job, releasing the GPU
qstat -u $USER              # confirm nothing of yours is still running
pgrep -af ollama            # confirm no stray server
```

Then close the SSH tunnel window on the laptop. Verify from the laptop that nothing is still held:

```
ssh chikhala@vega.erau.edu "qstat -u $USER; pgrep -af ollama"
```

An interactive job's walltime runs from when it starts, not when you arrive
If a queued interactive job begins overnight, it holds a GPU doing nothing until the walltime expires. Ask for a window you will actually be present for, and cancel and resubmit rather than letting one idle.

19 Open questions

Recorded because an unanswered question is part of the state of the work.

1. **How should a job read its GPU allocation?** Both `CUDA_VISIBLE_DEVICES` and `PBS_GPUFILE` were empty. Reading `nvidia-smi` and pinning works but relies on guessing correctly, and guessing wrong disturbs another user's job.
2. **Does `/scratch` get purged?** 47 GB of weights live there and it is unclear whether they must be re-staged.
3. **Is there a shared model cache?** Several users downloading the same 42 GB model separately wastes a great deal of space.
4. **Is a long-running inference server inside a job, reached by SSH forwarding, a supported pattern?** It works cleanly; confirmation is better than assumption.
5. **Is there a reservation mechanism for interactive work?** See the walltime warning above.
6. ~~**When will `gpuq` exist?**~~ **Answered 9 October 2026**: it exists, it works, and a request that waited two hours before the split now starts immediately. `batch` was disabled in the same change. Kept here because the sequence is the lesson — the restriction half of a change can land before the half that makes it usable, and for three days there was no documented route to a GPU at all.

A Command reference

A.1 Laptop, before anything else

```
Test-NetConnection vega.erau.edu -Port 22
```

A.2 On the login node

```
ssh chikhala@vega.erau.edu
hostname; echo $0; pwd; echo $HOME; echo $USER
df -h ~ /scratch
du -sh /scratch/$USER
source /scratch/$USER/ollama/env.sh
ollama --version
ollama list
```

A.3 Seeing the cluster

```
showq                       # everyone's jobs
showq -i                    # idle, in priority order
showq -b                    # blocked by policy
checkjob <jobid>            # why is this job not starting
qstat -u $USER
qstat -Q
qstat -f <jobid>
qdel <jobid>

pbsnodes -a | awk '/^[^ ]/{n=$1} /^ *state =/{print n, $3}'
pbsnodes -a | awk '/^ *state =/{print $3}' | sort | uniq -c
pbsnodes gpu01 gpu02 | grep -E "^gpu|state =|jobs ="
pbsnodes gpu01 | tr ',' '\n' | grep -E "ncpus|loadave|totmem|availmem"
```

A.4 Which cards are idle, on both GPU nodes

```
ssh gpu01 nvidia-smi --query-gpu=index,memory.used,utilization.gpu --format=csv
ssh gpu02 nvidia-smi --query-gpu=index,memory.used,utilization.gpu --format=csv

ssh gpu01 nvidia-smi --query-compute-apps=gpu_uuid,pid,process_name,used_memory --
    ↪ format=csv
ssh gpu02 nvidia-smi --query-compute-apps=gpu_uuid,pid,process_name,used_memory --
    ↪ format=csv
```

A.5 Getting a GPU

```
qstat -Q                    # which queues exist, and is Ena = yes
qsub -I -q gpuq -l nodes=1:ppn=48:gpus=1,mem=64gb,walltime=04:00:00
qsub -I -q gpuq -l nodes=gpu02:ppn=48:gpus=1,mem=64gb,walltime=04:00:00   # a named
    ↪ node
hostname                    # which node you got -- the tunnel must name it
```

A.6 Updating Ollama (login node, nothing serving)

```
source /scratch/$USER/ollama/env.sh
ollama --version
pgrep -af "ollama serve"                # must print nothing
curl -s https://api.github.com/repos/ollama/ollama/releases/latest | grep '"tag_name"'

cd /scratch/$USER/ollama
```

```
curl -L --progress-bar -o ollama-0.40.6.tar.zst \
  https://github.com/ollama/ollama/releases/download/v0.40.6/ollama-linux-amd64.tar.zst
ls -lh ollama-0.40.6.tar.zst        # ~1.44 GB
curl -sL -o sha256sum-0.40.6.txt \
  https://github.com/ollama/ollama/releases/download/v0.40.6/sha256sum.txt
grep -F "$(sha256sum ollama-0.40.6.tar.zst | cut -d' ' -f1)" sha256sum-0.40.6.txt &&
    ↪ echo VERIFIED

rm -rf bin lib                          # only after VERIFIED
zstd -dc ollama-0.40.6.tar.zst | tar -xf -
ollama --version && ls lib/ollama | grep -i cuda
du -sh models && ls models/manifests/registry.ollama.ai/library
rm -f ollama-0.40.6.tar.zst
```

A.7 On the GPU node

```
source /scratch/$USER/ollama/env.sh
hostname
nvidia-smi
export CUDA_VISIBLE_DEVICES=0

OLLAMA_HOST=0.0.0.0:11434 OLLAMA_CONTEXT_LENGTH=4096 \
  nohup ollama serve > /scratch/$USER/ollama/serve-gpu.log 2>&1 &

ollama run llama3.3:70b "reply with the single word: ready"
nvidia-smi
```

A.8 From the laptop

```
ssh -N -o ServerAliveInterval=30 -L 11435:gpu01:11434 chikhala@vega.erau.edu
# job on gpu02?  then:  -L 11435:gpu02:11434
```

```
curl -s http://127.0.0.1:11435/api/tags
curl -s http://127.0.0.1:11435/api/version    # the version the runner will record
```

```
conda activate carlaAir
cd $env:CARLAAIR_ROOT
.\CarlaAir.ps1 Town10HD
```

```
cd "<repository root>"
python scripts\run_missions.py --missions M01 --repeats 1 --ollama-host http
    ↪ ://127.0.0.1:11435
python scripts\run_missions.py --missions M01,M02,M03,M04,M05,M06,M07,M08 --block v4-
    ↪ polled --ollama-host http://127.0.0.1:11435
python scripts\run_missions.py --missions M09 --block v4-polled --ollama-host http
    ↪ ://127.0.0.1:11435
python scripts\run_missions.py --missions M10 --block v4-polled --ollama-host http
    ↪ ://127.0.0.1:11435
python tools\fly_chat.py
```

A.9 Shutting down

```
pkill -f "ollama serve"
exit
qstat -u $USER
```